Order Management Screen

Senior Frontend Developer: Take-Home Task

Overview

Build an order-management screen using **Vue 3**, the **Composition API**, and **TypeScript**. Use any supporting libraries you consider appropriate, and explain your choices.

Login

- Add a login page with email and password
- Create three test users, one for each role: Admin, Manager, and Support
- After login, load the user's role and permissions from `GET /me`
- Refreshing the page keeps the user logged in
- Users who aren't logged in are redirected to the login page, and a 401 response logs them out
- Add a logout option. Logging out and back in as a different user must not leave any of the previous user's data or permissions behind

Roles and Permissions

The app has three roles. Each role is granted a set of permissions:

Action	Permission	Admin	Manager	Support
View orders and order details	`orders.view`	✓	✓	✓
See order amounts	`orders.view_amount`	✓	✓	—
Create an order	`orders.create`	✓	✓	—
Edit an order	`orders.update`	✓	✓	—
Change an order's status	`orders.update_status`	✓	✓	—
Cancel an order	`orders.cancel`	✓	—	—
Access the Admin page	`admin.access`	✓	—	—

The `GET /me` endpoint returns the current user with their role and full list of permissions. Example response for a Manager:

```
{
  "id": 2,
  "name": "Sara Ali",
  "role": "manager",
  "permissions": [
    "orders.view",
    "orders.view_amount",
    "orders.create",
    "orders.update",
    "orders.update_status"
  ]
}
```

- Protect the Admin page (a simple placeholder is enough) so it can't be reached by typing its URL without permission

- The mock API must enforce the same rules: forbidden actions return 403, and order amounts are not sent to Support users. The UI must handle a 403 correctly, not rely only on hiding buttons
- Delivered and cancelled orders can't be edited or cancelled by anyone, including Admin
- Decide whether unavailable actions are hidden or disabled, and explain why
- Design the permission model so new roles or permissions are easy to add later

Mock API

Build the mock API yourself using `axios-mock-adapter`. It should:

- Serve about 50 generated orders, each with order number, customer, amount (IQD), date, and status (pending, processing, shipped, delivered, cancelled)
- Expose `POST /auth/login`, accepting email and password and returning a token. Wrong credentials return 401
- Expose `GET /me`, returning the current user, their role, and their permissions array
- Expose `GET /orders?q=&status=`, with search and filtering handled by the mock, not the UI
- Expose `GET /orders/:id`, returning the full order details including line items
- Expose `POST /orders` and `PUT /orders/:id` to create and edit orders. Invalid data returns 422 with an error message for each invalid field
- Expose `PATCH /orders/:id/status` and `POST /orders/:id/cancel`
- Enforce the permissions and the delivered/cancelled rule on every write endpoint
- Identify the current user from the token in the `Authorization` header. A missing or invalid token returns 401
- Add a random delay of 300–1000ms to each response
- Include an easy way to force errors (a flag or toggle) so error handling can be demonstrated

Screen Requirements

- Display orders with order number, customer, amount, date, and status
- Add customer search and a status filter, and make sure fast typing behaves correctly
- Open an order's details in a side panel or separate view
- Let users change an order's status and cancel an order from the details view, based on their permissions
- Handle loading, no results, and API errors, with a retry option
- Make the screen usable on mobile and fully usable with a keyboard, including opening and closing details

Create and Edit Orders

- Add a form to create a new order and edit an existing one, using the same form for both
- Fields: customer name, phone, address, and line items (product, quantity, unit price)
- Users can add and remove line items, and the order total is calculated automatically
- Validate the form before submitting, and show the API's 422 field errors under the matching fields
- After saving, the orders list and details show the updated data

Tests

Include at least two meaningful tests covering user behavior, runnable with `npm test`. At least one must cover a permission scenario.

Submission

Within **5 days**, send a GitHub repository with a short README covering:

- Setup instructions and the login details for the three test users
- How your permission model works and how it would connect to a real backend
- Key decisions and library choices
- Time spent
- What you would improve next
- Any AI assistance used